

```
"""
```

```
Growth Experiment Tracker
```

```
-----
```

```
A CLI tool to run, track, and auto-analyze growth experiments  
(social posts, SEO tests, email campaigns) using AI to decide  
winners vs losers automatically.
```

```
Philosophy: Test fast. Scale the winners. Kill the losers.
```

```
Author: Mahveen Qureshi
```

```
"""
```

```
import json  
import os  
import sys  
from datetime import datetime, timedelta  
from pathlib import Path  
import random
```

```
try:
```

```
    import anthropic  
    CLIENT = anthropic.Anthropic()  
    USE_CLAUDE = True
```

```
except Exception:
```

```
    USE_CLAUDE = False
```

```
DB_FILE = Path("experiments.json")
```

```
# — Database (flat JSON file) —————
```

```
def load_db() -> dict:
```

```
    if DB_FILE.exists():  
        return json.loads(DB_FILE.read_text())  
    return {"experiments": []}
```

```
def save_db(db: dict):
```

```
    DB_FILE.write_text(json.dumps(db, indent=2))
```

```
def next_id(db: dict) -> int:
```

```
    ids = [e["id"] for e in db["experiments"]]  
    return max(ids, default=0) + 1
```

```
# — Core Functions —————
```

```

def add_experiment(name: str, channel: str, hypothesis: str, metric: str):
    """Add a new growth experiment."""
    db = load_db()
    exp = {
        "id": next_id(db),
        "name": name,
        "channel": channel,          # social | seo | email | content
        "hypothesis": hypothesis,
        "metric": metric,            # e.g. "clicks", "signups", "open_rate"
        "status": "running",        # running | winner | killed
        "created_at": datetime.now().isoformat(),
        "results": []
    }
    db["experiments"].append(exp)
    save_db(db)
    print(f"✅ Experiment #{exp['id']} created: {name}")
    return exp


def log_result(exp_id: int, value: float, note: str = ""):
    """Log a daily result for an experiment."""
    db = load_db()
    exp = next((e for e in db["experiments"] if e["id"] == exp_id), None)
    if not exp:
        print(f"❌ Experiment #{exp_id} not found")
        return
    entry = {
        "date": datetime.now().strftime("%Y-%m-%d"),
        "value": value,
        "note": note
    }
    exp["results"].append(entry)
    save_db(db)
    print(f"🇮🇹 Logged {value} for '{exp['name']}'")


def ai_verdict(exp: dict) -> dict:
    """Use Claude to analyze results and give a SCALE/KILL verdict."""
    if not exp["results"]:
        return {"verdict": "INSUFFICIENT DATA", "reasoning": "No results logged y

    values = [r["value"] for r in exp["results"]]
    avg = sum(values) / len(values)
    trend = "improving" if len(values) > 1 and values[-1] > values[0] else "decli

    if USE_CLAUDE:
        prompt = f"""You are a growth analyst. Analyze this experiment and give a

```

```

Experiment: {exp['name']}
Channel: {exp['channel']}
Hypothesis: {exp['hypothesis']}
Metric: {exp['metric']}
Results over {len(values)} days: {values}
Average: {avg:.2f}
Trend: {trend}

```

Return a JSON object with:

- verdict: exactly "SCALE IT" or "KILL IT" or "KEEP TESTING"
- reasoning: one sentence why
- action: specific next step to take
- confidence: high/medium/low

Return ONLY valid JSON."""

```

try:
    msg = CLIENT.messages.create(
        model="claude-opus-4-5",
        max_tokens=256,
        messages=[{"role": "user", "content": prompt}]
    )
    return json.loads(msg.content[0].text)
except Exception:
    pass

```

Demo mode heuristic

if avg > 100 and trend == "improving":

```
    return {"verdict": "SCALE IT", "reasoning": f"Strong avg of {avg:.0f} wit
```

elif avg < 20 or trend == "declining":

```
    return {"verdict": "KILL IT", "reasoning": f"Avg of {avg:.0f} is below th
```

else:

```
    return {"verdict": "KEEP TESTING", "reasoning": f"Avg of {avg:.0f} - too
```

```
def analyze_all():
```

```
    """Analyze all running experiments and print verdicts."""
```

```
    db = load_db()
```

```
    running = [e for e in db["experiments"] if e["status"] == "running"]
```

```
    if not running:
```

```
        print("No running experiments. Add one with: python tracker.py add")
```

```
        return
```

```
    print(f"\n{'='*55}")
```

```
    print(f"    GROWTH EXPERIMENT ANALYSIS - {datetime.now().strftime('%Y-%m-%d')}")
```

```
    print(f"{'='*55}")
```

```

for exp in running:
    print(f"\n📌 #{exp['id']} {exp['name']} [{exp['channel'].upper()}]")
    print(f"    Hypothesis: {exp['hypothesis']}")
    if exp["results"]:
        values = [r["value"] for r in exp["results"]]
        print(f"    Data points: {len(values)} | Avg: {sum(values)/len(values)}")
    verdict = ai_verdict(exp)
    emoji = "🚀" if "SCALE" in verdict["verdict"] else "💀" if "KILL" in verdict["verdict"]
    print(f"    {emoji} VERDICT: {verdict['verdict']} [{verdict.get('confidence', 0)}%]")
    print(f"    → {verdict['reasoning']}")
    print(f"    → ACTION: {verdict['action']}")

    # Auto-update status
    if "SCALE" in verdict["verdict"]:
        exp["status"] = "winner"
    elif "KILL" in verdict["verdict"]:
        exp["status"] = "killed"
    save_db(db)

# Summary
db2 = load_db()
winners = sum(1 for e in db2["experiments"] if e["status"] == "winner")
killed = sum(1 for e in db2["experiments"] if e["status"] == "killed")
print(f"\n{'='*55}")
print(f"    SUMMARY: {winners} winners | {killed} killed | {len(running)} analyzing")
print(f"{'='*55}\n")

def seed_demo():
    """Seed realistic demo data so the tracker shows real output."""
    db = load_db()
    if db["experiments"]:
        print("Demo data already exists. Run: python tracker.py analyze")
        return

    experiments = [
        ("LinkedIn founder story post", "social", "Personal story posts get 3x more engagement"),
        ("SEO blog: AI for Amazon sellers", "seo", "Long-tail keyword blog will rank for high intent"),
        ("Cold email: agency owners", "email", "Pain-focused subject lines get >40% open rate")
    ]

    results_data = [
        [120, 340, 890, 1200, 2100], # viral - winner
        [8, 12, 9, 11, 10], # flat - kill it
        [38, 41, 45, 43, 48], # good open rate - keep testing
    ]

```

```

    for (name, channel, hyp, metric), results in zip(experiments, results_data):
        exp = add_experiment(name, channel, hyp, metric)
        for i, val in enumerate(results):
            date = (datetime.now() - timedelta(days=len(results)-i-1)).strftime("%Y-%m-%d")
            exp["results"].append({"date": date, "value": val, "note": ""})

save_db({"experiments": db["experiments"] + [e for e in load_db()["experiment"]])
print("\n✅ Demo data seeded. Now run: python tracker.py analyze")

def show_help():
    print("""
Growth Experiment Tracker
Usage:
python tracker.py demo          # Load demo data and analyze
python tracker.py analyze       # Analyze all running experiments
python tracker.py add           # Add a new experiment (interactive)
python tracker.py log <id> <value> # Log a result for an experiment
python tracker.py list          # List all experiments
""")

def list_all():
    db = load_db()
    if not db["experiments"]:
        print("No experiments yet. Run: python tracker.py demo")
        return
    print(f"\n{'ID':<4} {'Name':<35} {'Channel':<10} {'Status':<12} {'Results':<12}")
    print("-" * 75)
    for e in db["experiments"]:
        print(f"#{e['id']:<3} {e['name'][:34]:<35} {e['channel']:<10} {e['status'][:12]:<12} {e['results'][:12]:<12}")

# — CLI —————

if __name__ == "__main__":
    cmd = sys.argv[1] if len(sys.argv) > 1 else "help"

    if cmd == "demo":
        seed_demo()
        analyze_all()

    elif cmd == "analyze":
        analyze_all()

    elif cmd == "list":

```

```
list_all()

elif cmd == "log":
    if len(sys.argv) < 4:
        print("Usage: python tracker.py log <experiment_id> <value>")
    else:
        log_result(int(sys.argv[2]), float(sys.argv[3]))

elif cmd == "add":
    print("New Experiment")
    name = input("Name: ")
    channel = input("Channel (social/seo/email/content): ")
    hypothesis = input("Hypothesis: ")
    metric = input("Metric to track (e.g. clicks, signups): ")
    add_experiment(name, channel, hypothesis, metric)

else:
    show_help()
```